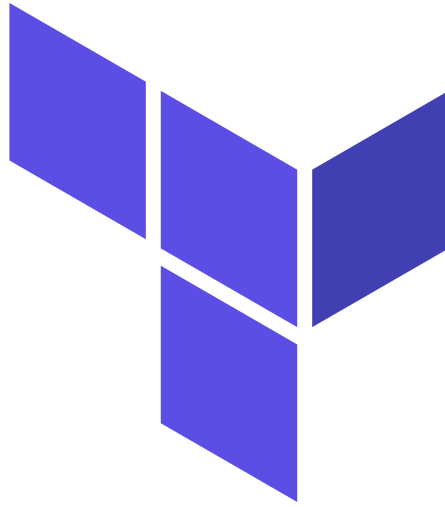




Apuntes Terraform: De Cero a Master

Guía profesional, práctica y escalable

Objetivo: dominar Terraform en escenarios reales: cloud, on-premise, Kubernetes, CI/CD, seguridad, gobernanza y operación enterprise.

DevOps Notes

17 de marzo de 2026

Índice

1. Cómo usar estos apuntes (ruta recomendada)	4
1.1. Ruta por niveles	4
1.2. Competencias que debes dominar	4
2. Qué es Terraform y por qué importa	4
2.1. Problemas que resuelve	4
2.2. Conceptos base	4
3. Terraform en on-premise y entornos híbridos	5
3.1. Casos on-premise frecuentes	5
3.2. On-prem + cloud (híbrido) bien diseñado	5
3.3. Blueprint visual de arquitectura híbrida	6
4. Instalación, estructura mínima y flujo de trabajo	6
4.1. Instalación rápida	6
4.2. Ciclo de vida básico (imprescindible)	6
4.3. Estructura mínima recomendada	7
5. HCL a fondo (HashiCorp Configuration Language)	7
5.1. Bloques y atributos	7
5.2. Variables, tipos y validaciones	7
5.3. Locals y funciones	8
6. State: el corazón de Terraform	8
6.1. Qué contiene el state	8
6.2. Buenas prácticas críticas	8
6.3. Backend remoto en AWS (ejemplo de producción)	9
7. Dependencias, meta-argumentos y control avanzado	9
7.1. count vs for_each	9
7.2. Meta-argumentos clave	9
8. Módulos: de básico a diseño enterprise	10
8.1. Qué es un módulo	10
8.2. Estructura recomendada de módulo	10
8.3. Consumo de módulo	10
8.4. Principios de módulos de calidad	11
9. Workspaces, entornos y estrategia multi-ambiente	11
9.1. Cuándo usar workspaces	11
9.2. Patrón recomendado en equipos grandes	11
10. Seguridad real en Terraform	11
10.1. Gestión de secretos	11
10.2. Políticas y compliance	12
11. Testing y calidad profesional	12
11.1. Niveles de validación	12

12.Import, refactor y migraciones sin downtime	12
12.1. Importar recursos existentes	12
12.2. Refactor seguro con moved blocks	12
13.CI/CD con Terraform (modelo recomendado)	13
13.1. Flujo empresarial	13
14.Integración profunda con Kubernetes (EKS, AKS, GKE)	13
14.1. Enfoque recomendado: Terraform para plataforma, GitOps para apps	13
14.2. Providers típicos en ecosistema Kubernetes	13
14.3. Instalación de add-ons con Helm provider	14
14.4. Patrones avanzados en Kubernetes + Terraform	14
14.5. Errores típicos en k8s con Terraform	15
14.6. Kubernetes on-prem (RKE2, K3s, OpenShift, vanilla)	15
15.Arquitectura de plataforma: landing zone y multi-cuenta	15
15.1. Modelo de referencia en cloud pública	15
15.2. Estrategia por stacks	15
15.3. Terragrunt (opcional, muy usado en enterprise)	16
15.4. Blueprint de repositorio enterprise	16
16.Patrones de seguridad avanzados	16
16.1. Identidades efímeras en CI/CD	16
16.2. Cifrado, secretos y datos sensibles	17
16.3. Policy as Code obligatoria	17
17.Optimización de costes con Terraform	17
17.1. Prácticas de FinOps aplicadas a IaC	17
17.2. Ejemplo de etiquetas obligatorias por variable	17
18.Rendimiento y escalabilidad de Terraform	17
18.1. Cuellos de botella habituales	17
18.2. Cómo escalar sin dolor	18
19.Gestión avanzada del state (operaciones delicadas)	18
19.1. Comandos de state que debes dominar con cuidado	18
19.2. Reglas de oro antes de tocar state manualmente	18
20.Provisioners y alternativas modernas	18
20.1. Por qué evitar provisioners cuando puedas	18
20.2. Alternativas preferibles	18
21.Estrategias de migración a Terraform desde infra existente	19
21.1. Plan de migración por fases	19
21.2. Matriz de riesgo recomendada	19
22.Terraform Cloud/Enterprise y operación colaborativa	19
22.1. Cuándo usar Terraform Cloud	19
22.2. Conceptos clave	19

23.Patrones multi-cloud y disaster recovery	19
23.1. Multi-cloud realista	19
23.2. DR como código	20
24.Observabilidad de la infraestructura gestionada por Terraform	20
24.1. Qué observar	20
24.2. Stack típico	20
25.Anti-patrones que debes evitar	20
26.Laboratorios guiados (de básico a experto)	20
27.Checklist de salida a producción (Go-Live)	21
27.1. Runbook mínimo de incidente IaC	21
28.Troubleshooting experto	21
28.1. Errores comunes y solución	21
28.2. Comandos de diagnóstico	22
29.Buenas prácticas “de senior a master”	22
30.Roadmap de estudio: de 0 a master	22
31.Cheat Sheet final	23
32.Glosario rápido (términos clave)	23
33.Conclusión	24

1 Cómo usar estos apuntes (ruta recomendada)

Modo de estudio sugerido: cada sección debe ir acompañada de práctica real. Regla simple: *leer 30 %, practicar 70 %*.

1.1 Ruta por niveles

Nivel	Objetivo práctico
Inicial	Crear infra mínima, entender state, usar <code>plan/apply/destroy</code> sin errores.
Intermedio	Diseñar módulos reutilizables, backend remoto, CI con <code>plan</code> en PR.
Avanzado	Operar multi-entorno/multi-cuenta, seguridad IaC y políticas obligatorias.
Master	Diseñar plataforma completa (cloud + on-prem + Kubernetes) con gobernanza y observabilidad.

1.2 Competencias que debes dominar

- Modelado declarativo y diseño de módulos mantenibles.
- Operación segura del state y migraciones sin downtime.
- Integración con Kubernetes y GitOps en entornos reales.
- Automatización enterprise (CI/CD, policy as code, FinOps, DR).

2 Qué es Terraform y por qué importa

Terraform es una herramienta de **Infraestructura como Código (IaC)** creada por HashiCorp. Permite describir infraestructura en archivos declarativos y aplicarla de forma reproducible.

2.1 Problemas que resuelve

- Evita cambios manuales no trazables (*clickops*).
- Estandariza despliegues entre entornos (dev, qa, prod).
- Facilita revisión por *pull request* y auditoría.
- Reduce errores humanos y acelera entregas.

2.2 Conceptos base

- **Provider:** plugin que traduce Terraform a API cloud (AWS, Azure, GCP, Kubernetes, etc.).
- **Resource:** bloque que crea/modifica/elimina un objeto real.
- **Data source:** lee objetos existentes sin crearlos.

- **State:** archivo que guarda el mapeo entre código y recursos reales.
- **Plan:** diferencia entre estado actual y estado deseado.

3 Terraform en on-premise y entornos híbridos

Sí, Terraform se puede usar **perfectamente en on-premise**. No está limitado a nube pública. Terraform funciona contra cualquier plataforma con provider compatible o API integrable.

3.1 Casos on-premise frecuentes

- **VMware vSphere:** VMs, datastores, redes virtuales.
- **OpenStack:** redes, compute, volúmenes.
- **Proxmox:** virtualización y recursos de clúster.
- **Kubernetes on-prem:** namespaces, RBAC, Helm, add-ons.
- **DNS/IPAM/firewall:** Infoblox, F5, Palo Alto, Fortinet, etc.

```
terraform {
  required_providers {
    vsphere = {
      source  = "hashicorp/vsphere"
      version = "~> 2.5"
    }
  }
}

provider "vsphere" {
  user           = var.vsphere_user
  password       = var.vsphere_password
  vsphere_server = var.vsphere_server
  allow_unverified_ssl = true
}

resource "vsphere_virtual_machine" "app01" {
  name              = "app01"
  resource_pool_id  = data.vsphere_resource_pool.pool.id
  datastore_id      = data.vsphere_datastore.datastore.id
  num_cpus          = 2
  memory            = 4096
  guest_id          = "ubuntu64Guest"
}
```

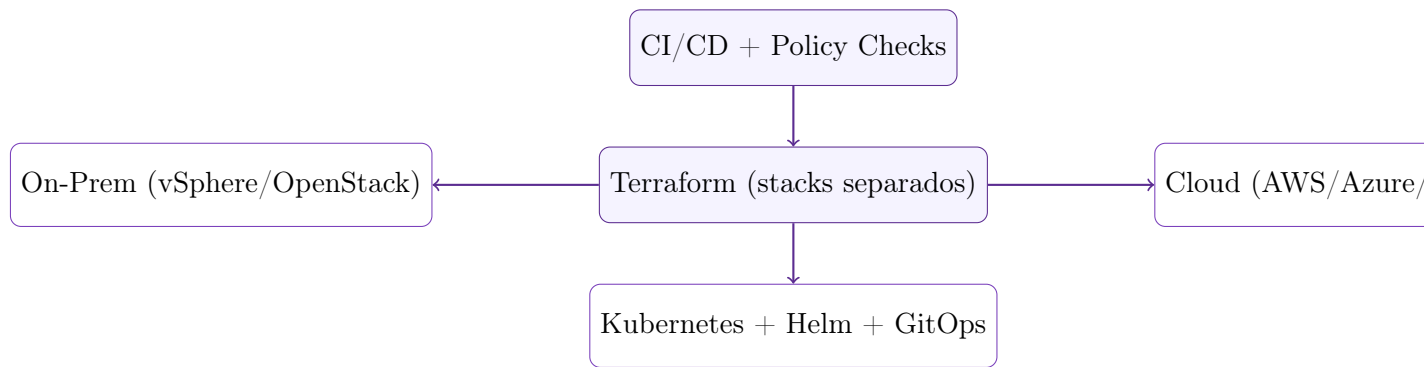
Listing 1: Ejemplo básico on-prem con vSphere

3.2 On-prem + cloud (híbrido) bien diseñado

1. Estados separados por dominio (on-prem network, cloud network, k8s, apps-bootstrap).

2. Conectividad explícita (VPN/Direct Connect/ExpressRoute + routing + DNS).
3. Mismos estándares de tags, naming, políticas y revisiones.
4. Pipeline única con aprobaciones por entorno.

3.3 Blueprint visual de arquitectura híbrida



Regla práctica: Terraform provisiona la plataforma (infra), mientras GitOps y herramientas de configuración gestionan el ciclo de vida dinámico de aplicaciones.

4 Instalación, estructura mínima y flujo de trabajo

4.1 Instalación rápida

```
# Linux (ejemplo con apt y repositorio oficial)
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
sudo apt-add-repository "deb [arch=amd64] https://apt.releases.hashicorp.com $(lsb_release -cs) main"
sudo apt-get update && sudo apt-get install terraform

terraform -version
```

Listing 2: Instalación y verificación

4.2 Ciclo de vida básico (imprescindible)

```
terraform init      # Descarga providers y prepara backend
terraform fmt       # Formatea el código
terraform validate  # Valida sintaxis y coherencia
terraform plan      # Muestra cambios previstos
terraform apply     # Aplica los cambios
terraform destroy   # Elimina lo gestionado
```

Listing 3: Comandos esenciales

4.3 Estructura mínima recomendada

```
terraform-project/  
|- -- main.tf  
|- -- variables.tf  
|- -- outputs.tf  
|- -- providers.tf  
|- -- versions.tf  
+- -- terraform.tfvars
```

Listing 4: Estructura de proyecto

5 HCL a fondo (HashiCorp Configuration Language)

5.1 Bloques y atributos

```
terraform {  
  required_version = ">= 1.6.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = var.aws_region  
}  
  
resource "aws_s3_bucket" "logs" {  
  bucket = "mi-bucket-logs-unico-12345"  
  tags = {  
    Project = "plataforma"  
    Owner   = "devops"  
  }  
}
```

Listing 5: Ejemplo base en HCL

5.2 Variables, tipos y validaciones

```
variable "aws_region" {  
  description = "Región de AWS"  
  type        = string  
  default     = "eu-west-1"  
}  
  
variable "instance_count" {  
  description = "Número de instancias"
```



```
type          = number
default       = 2

validation {
  condition    = var.instance_count >= 1 && var.instance_count <=
    10
  error_message = "instance_count debe estar entre 1 y 10."
}

}

variable "common_tags" {
  type = map(string)
  default = {
    Environment = "dev"
    Team        = "platform"
  }
}
```

Listing 6: Variables tipadas y validadas

5.3 Locals y funciones

```
locals {
  name_prefix = "${var.project}-${var.environment}"
  merged_tags = merge(var.common_tags, {
    ManagedBy = "terraform"
  })
}

# Ejemplos de funciones útiles
# lower(), upper(), format(), join(), split(), lookup(), try(), can
# (), coalesce()
```

Listing 7: Locals para evitar duplicación

6 State: el corazón de Terraform

6.1 Qué contiene el state

El `terraform.tfstate` guarda IDs reales, dependencias y metadatos. Si se pierde o corrompe, Terraform puede intentar recrear recursos que ya existen.

6.2 Buenas prácticas críticas

- Nunca comitear state local en Git.
- Usar backend remoto con bloqueo (S3 + DynamoDB, Azure Storage, GCS, Terraform Cloud).
- Hacer copias de seguridad y control de acceso estricto.

6.3 Backend remoto en AWS (ejemplo de producción)

```
terraform {  
  backend "s3" {  
    bucket      = "tfstate-company-prod"  
    key         = "network/prod/terraform.tfstate"  
    region      = "eu-west-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

Listing 8: Backend S3 con locking en DynamoDB

7 Dependencias, meta-argumentos y control avanzado

7.1 count vs for_each

- **count**: útil para colecciones indexadas simples.
- **for_each**: preferido para colecciones con clave estable (menos drift al cambiar orden).

```
variable "subnets" {  
  type = map(object({  
    cidr = string  
    az   = string  
  }))  
}  
  
resource "aws_subnet" "this" {  
  for_each      = var.subnets  
  vpc_id        = aws_vpc.main.id  
  cidr_block    = each.value.cidr  
  availability_zone = each.value.az  
  
  tags = {  
    Name = "subnet-${each.key}"  
  }  
}
```

Listing 9: for_each recomendado

7.2 Meta-argumentos clave

Meta-argumento	Uso principal
depends_on	Fuerza dependencia explícita cuando Terraform no la infiere.
lifecycle	Controla creación/destrucción y evita borrados accidentales.
provider	Permite seleccionar provider/alias específico.
count	Repite recurso por número.

<code>for_each</code>	Repite recurso por claves de mapa/set.
-----------------------	--

```
resource "aws_launch_template" "app" {
  name_prefix    = "app-"
  image_id       = data.aws_ami.ubuntu.id
  instance_type  = "t3.micro"

  lifecycle {
    create_before_destroy = true
  }
}
```

Listing 10: lifecycle para minimizar downtime

8 Módulos: de básico a diseño enterprise

8.1 Qué es un módulo

Un módulo es una unidad reutilizable de infraestructura. Todo código Terraform está en un módulo; el directorio raíz es el *root module*.

8.2 Estructura recomendada de módulo

```
modules/
+- -- vpc/
    |- -- main.tf
    |- -- variables.tf
    |- -- outputs.tf
    +- -- README.md
```

Listing 11: Estructura de módulo reutilizable

8.3 Consumo de módulo

```
module "network" {
  source      = "../modules/vpc"
  project     = var.project
  environment = var.environment
  vpc_cidr    = "10.20.0.0/16"
  subnet_cidrs = ["10.20.1.0/24", "10.20.2.0/24"]
}

output "vpc_id" {
  value = module.network.vpc_id
}
```

Listing 12: Uso de módulo local

8.4 Principios de módulos de calidad

1. Interfaz clara (variables y outputs mínimos).
2. Encapsulación (sin exponer detalles internos innecesarios).
3. Versionado semántico y changelog.
4. Tests y ejemplos de uso.
5. Compatibilidad retroactiva en releases mayores/menores.

9 Workspaces, entornos y estrategia multi-ambiente

9.1 Cuándo usar workspaces

Útiles para diferencias ligeras entre entornos. Para diferencias estructurales grandes, preferir directorios/separación por stacks.

```
terraform workspace list
terraform workspace new dev
terraform workspace new prod
terraform workspace select prod
```

Listing 13: Comandos de workspaces

9.2 Patrón recomendado en equipos grandes

- Separar por capas: `network`, `security`, `data`, `apps`.
- Un state por stack y entorno.
- Promoción por PR y pipeline, nunca cambios manuales en prod.

10 Seguridad real en Terraform

10.1 Gestión de secretos

- No guardar secretos en variables planas ni en `.tfvars` versionados.
- Integrar con Secrets Manager, Vault o SSM Parameter Store.
- Marcar outputs sensibles.

```
output "db_password" {
  value      = random_password.db.result
  sensitive = true
}
```

Listing 14: Output sensible

10.2 Políticas y compliance

- **tflint**: calidad y convenciones.
- **tfsec/checkov**: seguridad estática IaC.
- **OPA/Conftest** o **Sentinel**: políticas obligatorias (guardrails).

11 Testing y calidad profesional

11.1 Niveles de validación

1. **Formato y sintaxis**: terraform fmt -check, terraform validate.
2. **Análisis estático**: tflint, tfsec, checkov.
3. **Tests de integración**: Terratest (Go) o pipelines efímeros.

```
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
tflint
tfsec .
```

Listing 15: Pipeline mínima de calidad

12 Import, refactor y migraciones sin downtime

12.1 Importar recursos existentes

```
# Declaras primero el bloque resource
resource "aws_s3_bucket" "legacy" {
  bucket = "bucket-existente"
}

# Luego importas su estado
terraform import aws_s3_bucket.legacy bucket-existente
```

Listing 16: Import básico

12.2 Refactor seguro con moved blocks

```
moved {
  from = aws_instance.web
  to   = aws_instance.frontend
}
```

Listing 17: Renombrado sin recrear recursos

13 CI/CD con Terraform (modelo recomendado)

13.1 Flujo empresarial

1. Pull Request ejecuta `fmt`, `validate`, seguridad y `plan`.
2. Aprobación obligatoria de al menos un revisor.
3. `apply` solo en rama protegida y con credenciales de corta duración.
4. Registro de artefacto `plan` y auditoría de cambios.

```
stages:
  - lint
  - validate
  - plan
  - apply

plan:
  script:
    - terraform init
    - terraform plan -out=tfplan

apply:
  when: manual
  script:
    - terraform apply -auto-approve tfplan
```

Listing 18: Esqueleto CI (pseudo-pipeline)

14 Integración profunda con Kubernetes (EKS, AKS, GKE)

14.1 Enfoque recomendado: Terraform para plataforma, GitOps para apps

Terraform destaca creando **infraestructura de clúster** (red, nodos, IAM, políticas, ingress base, observabilidad base). Para despliegue continuo de aplicaciones, suele ser mejor GitOps (Argo CD / Flux) para evitar planes gigantes y drift frecuente.

14.2 Providers típicos en ecosistema Kubernetes

- **hashicorp/kubernetes**: gestiona recursos Kubernetes desde Terraform.
- **hashicorp/helm**: instala charts Helm de forma declarativa.
- **hashicorp/aws**, **hashicorp/azurerm**, **hashicorp/google**: crean clúster administrado y recursos cloud.

```
provider "aws" {
  region = var.aws_region
}
```

```
data "aws_eks_cluster" "this" {
  name = module.eks.cluster_name
}

data "aws_eks_cluster_auth" "this" {
  name = module.eks.cluster_name
}

provider "kubernetes" {
  host = data.aws_eks_cluster.this.endpoint
  cluster_ca_certificate = base64decode(data.aws_eks_cluster.this.
    certificate_authority[0].data)
  token = data.aws_eks_cluster_auth.this.token
}

resource "kubernetes_namespace" "platform" {
  metadata {
    name = "platform"
  }
}
```

Listing 19: Ejemplo mínimo: EKS + provider kubernetes

14.3 Instalación de add-ons con Helm provider

```
resource "helm_release" "ingress_nginx" {
  name      = "ingress-nginx"
  repository = "https://kubernetes.github.io/ingress-nginx"
  chart     = "ingress-nginx"
  namespace = "ingress-nginx"

  create_namespace = true
  atomic           = true
  timeout          = 900

  set {
    name  = "controller.replicaCount"
    value = "2"
  }
}
```

Listing 20: Ingress NGINX con Helm

14.4 Patrones avanzados en Kubernetes + Terraform

1. Crear clúster y networking con Terraform.
2. Instalar add-ons base (ingress, cert-manager, external-dns, metrics-server).
3. Delegar aplicaciones al flujo GitOps.
4. Separar states: cluster, addons, apps-bootstrap.

14.5 Errores típicos en k8s con Terraform

- **Race conditions:** el API server aún no está listo tras crear clúster.
- **Token expirado:** providers que usan auth temporal en pipelines largos.
- **Diff infinito en helm:** valores no deterministas o charts con defaults variables.
- **Drift alto:** si alguien aplica con kubectl/helm fuera de Terraform.

14.6 Kubernetes on-prem (RKE2, K3s, OpenShift, vanilla)

- Terraform puede aprovisionar VMs/red on-prem y bootstrap del clúster.
- Para OpenShift, suele coexistir con instaladores nativos + Terraform para infraestructura periférica.
- El patrón ganador sigue siendo: **Terraform para base, GitOps para workloads.**

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}

resource "kubernetes_namespace" "observability" {
  metadata { name = "observability" }
}

resource "helm_release" "kube_prometheus_stack" {
  name          = "kube-prometheus-stack"
  repository    = "https://prometheus-community.github.io/helm-charts"
  chart         = "kube-prometheus-stack"
  namespace     = kubernetes_namespace.observability.metadata[0].name
}
```

Listing 21: Patrón on-prem k8s: namespaces base + helm add-ons

15 Arquitectura de plataforma: landing zone y multi-cuenta

15.1 Modelo de referencia en cloud pública

- Cuenta/suscripción de **seguridad** (logs, auditoría, KMS/HSM).
- Cuenta/suscripción de **red compartida** (transit gateway, firewall, DNS).
- Cuentas por **entorno** (dev/qa/prod) y por dominio de negocio.
- Identity centralizada con roles de acceso temporal.

15.2 Estrategia por stacks

```
live/
|- -- prod/
|   |- -- org-security/
|   |- -- shared-network/
```



```

|   |- -- eks-platform/
|   +- -- apps-bootstrap/
+- -- dev/
|   |- -- org-security/
|   |- -- shared-network/
|   |- -- eks-platform/
+- -- apps-bootstrap/

```

Listing 22: Separación por stacks y estados

15.3 Terragrunt (opcional, muy usado en enterprise)

Terragrunt ayuda a reducir duplicación (backend/provider/inputs repetidos), pero añade una capa operativa adicional. Úsalo si tu organización ya tiene muchos stacks y necesidad de DRY extremo.

15.4 Blueprint de repositorio enterprise

```

platform-iac/
|- -- modules/
|   |- -- network/
|   |- -- kubernetes/
|   |- -- observability/
|   +- -- security-baseline/
|- -- live/
|   |- -- prod/
|   |   |- -- shared-network/
|   |   |- -- eks-platform/
|   |   +- -- addons/
|   +- -- dev/
|- -- policies/
|   |- -- opa/
|   +- -- sentinel/
|- -- scripts/
|- -- docs/
+- -- .github/workflows/

```

Listing 23: Estructura recomendada para equipos grandes

16 Patrones de seguridad avanzados

16.1 Identidades efímeras en CI/CD

Evita claves estáticas. Usa OIDC entre tu plataforma CI y cloud provider.

```

# 1) CI obtiene token OIDC firmado
# 2) Cloud IAM valida token y emite credenciales temporales
# 3) Terraform usa esas credenciales para plan/apply

```

Listing 24: Idea de flujo seguro con OIDC

16.2 Cifrado, secretos y datos sensibles

- Cifrar backend y snapshots de state.
- Restringir acceso al state por principio de mínimo privilegio.
- Enmascarar variables sensibles en logs de pipeline.
- Rotación periódica de secretos y llaves KMS.

16.3 Policy as Code obligatoria

- Reglas de seguridad: no permitir buckets públicos, puertos críticos abiertos, etc.
- Reglas de gobierno: naming, tags, regiones permitidas.
- Reglas de coste: tamaños máximos de instancias o límites por entorno.

17 Optimización de costes con Terraform

17.1 Prácticas de FinOps aplicadas a IaC

1. Etiquetado obligatorio para `cost-center`, `owner`, `environment`.
2. Ambientes efímeros con TTL automático.
3. Apagado programado de recursos no productivos.
4. Revisión continua de derechos de cómputo (rightsizing).

17.2 Ejemplo de etiquetas obligatorias por variable

```
variable "mandatory_tags" {
  type = map(string)
  validation {
    condition = alltrue([
      contains(keys(var.mandatory_tags), "owner"),
      contains(keys(var.mandatory_tags), "cost-center"),
      contains(keys(var.mandatory_tags), "environment")
    ])
    error_message = "Faltan tags obligatorias: owner, cost-center,
      environment"
  }
}
```

Listing 25: Mapa de tags obligatorio

18 Rendimiento y escalabilidad de Terraform

18.1 Cuellos de botella habituales

- States gigantes con miles de recursos en un único stack.
- Dependencias en cadena que serializan operaciones.

- Provider con límites de API y throttling.

18.2 Cómo escalar sin dolor

- Dividir en stacks con ownership claro.
- Ejecutar planes en paralelo por dominio cuando sea seguro.
- Cachear plugins/providers en runners CI.
- Evitar data sources excesivos que consultan APIs en cada plan.

19 Gestión avanzada del state (operaciones delicadas)

19.1 Comandos de state que debes dominar con cuidado

```
terraform state list
terraform state show module.network.aws_vpc.main
terraform state mv old.address new.address
terraform state rm aws_s3_bucket.temp
terraform state pull
```

Listing 26: State operations

19.2 Reglas de oro antes de tocar state manualmente

1. Haz backup del state.
2. Ejecuta en ventana controlada y con revisión por pares.
3. Documenta el cambio en PR/incidencia.
4. Lanza plan inmediatamente después para validar coherencia.

20 Provisioners y alternativas modernas

20.1 Por qué evitar provisioners cuando puedas

`remote-exec` y `local-exec` son útiles en casos excepcionales, pero aumentan acoplamiento y fragilidad.

20.2 Alternativas preferibles

- **Cloud-init**/user-data para bootstrap de máquinas.
- Imágenes inmutables (Packer).
- Configuración con Ansible/SSM después del aprovisionamiento.
- Operadores/controladores en Kubernetes para configuración runtime.

21 Estrategias de migración a Terraform desde infra existente

21.1 Plan de migración por fases

1. Inventario de recursos críticos y dependencias.
2. Import por dominios pequeños (network primero, luego compute/data).
3. Normalización de naming/tags sin recreación.
4. Establecer pipeline obligatoria antes de ampliar alcance.

21.2 Matriz de riesgo recomendada

Tipo de recurso	Riesgo	Mitigación
Red core (VPC/VNet, rutas)	Alto	Entorno réplica, revisión senior, ventana de cambio.
Bases de datos	Alto	Backups, réplica, pruebas restore, plan de rollback.
Compute stateless	Medio	Despliegue blue/green y health checks.
Buckets/objetos	Medio	Versioning, bloqueo de borrado, políticas de retención.

22 Terraform Cloud/Enterprise y operación colaborativa

22.1 Cuándo usar Terraform Cloud

- Equipos grandes que requieren ejecución remota centralizada.
- Política de aprobación, colas de ejecución y control RBAC unificado.
- Integración nativa con policy checks y run tasks.

22.2 Conceptos clave

- **Workspace**: unidad de ejecución y state.
- **Run**: pipeline de plan/apply.
- **Variable sets**: variables compartidas por múltiples workspaces.
- **Policy checks**: bloqueo de cambios que violen reglas.

23 Patrones multi-cloud y disaster recovery

23.1 Multi-cloud realista

No todo debe ser multi-cloud activo-activo. Suele funcionar mejor:

- Cloud principal para producción.
- Cloud secundaria para servicios concretos o contingencia.
- Abstracción por módulos para evitar lock-in excesivo sin perder eficiencia.

23.2 DR como código

1. Definir RTO/RPO por servicio.
2. Versionar infraestructura de failover.
3. Ejecutar simulacros periódicos (*game days*).
4. Medir tiempos reales de recuperación y ajustar diseño.

24 Observabilidad de la infraestructura gestionada por Terraform

24.1 Qué observar

- Cambios de recursos por despliegue (auditoría de plan/apply).
- Eventos de seguridad y cambios de IAM/políticas.
- Coste incremental por release.
- SLO de plataforma (latencia API, errores, saturación).

24.2 Stack típico

- Logs/auditoría cloud nativos (CloudTrail, Activity Logs, Audit Logs).
- Métricas y alertas (Prometheus/Grafana o servicios gestionados).
- Trazabilidad de cambios vía PR + run IDs de Terraform.

25 Anti-patrones que debes evitar

1. Un único state para toda la empresa.
2. Mezclar infraestructura crítica y apps volátiles en el mismo stack.
3. Variables sin tipo ni validación.
4. Uso indiscriminado de `-target` en producción.
5. Aplicar en local sin trazabilidad ni aprobación.
6. Confiar solo en `terraform destroy` sin políticas de protección.

26 Laboratorios guiados (de básico a experto)

Lab 1: Fundamentos

Crear VPC/VNet + instancia + bucket con tags, outputs y variables tipadas.

Lab 2: Módulos y backend remoto

Extraer red a módulo, configurar backend remoto con locking, y separar dev/prod.

Lab 3: Kubernetes plataforma

Levantar EKS/AKS/GKE, instalar ingress + cert-manager con Helm provider.

Lab 4: Seguridad y políticas

Integrar tfint + tfsec + checkov + OPA en PR, bloquear merges inseguros.

Lab 5: Migración controlada

Importar infraestructura existente y refactorizar con `moved` sin downtime.

27 Checklist de salida a producción (Go-Live)

Checklist de producción

- Backend remoto cifrado, con locking y backups verificados.
- `fmt`, `validate`, `tfint`, `tfsec`/`checkov` en PR obligatorios.
- Política de aprobaciones activa (mínimo 1–2 revisores según criticidad).
- Credenciales efímeras (OIDC), sin secretos estáticos en CI.
- Runbook de rollback documentado y probado.
- Monitorización/alertas activas para recursos críticos.
- Matriz de ownership clara por stack (quién aprueba, quién opera, quién responde).

27.1 Runbook mínimo de incidente IaC

1. Congelar applies no urgentes.
2. Ejecutar `plan` contra estado actual y capturar evidencia.
3. Identificar si el problema es drift, permisos, dependencia o provider.
4. Aplicar mitigación segura (`rollback`, `state mv/rm`, `import`, `hotfix` en módulo).
5. Publicar postmortem con acciones preventivas.

28 Troubleshooting experto

28.1 Errores comunes y solución

Problema	Acción recomendada
----------	--------------------

Drift entre cloud y código	Ejecutar <code>plan</code> , revisar cambios manuales, reimportar si procede.
State bloqueado	Verificar proceso activo y liberar lock de forma segura.
Recreación inesperada de recurso	Revisar atributos <i>ForceNew</i> , usar <code>lifecycle</code> o rediseñar módulo.
Dependencias cíclicas	Romper ciclo con outputs intermedios o separación de stacks.

28.2 Comandos de diagnóstico

```
terraform state list
terraform state show aws_instance.web
terraform providers
terraform graph
TF_LOG=INFO terraform plan
```

Listing 27: Comandos útiles de depuración

29 Buenas prácticas “de senior a master”

1. Diseña módulos pequeños, versionados y bien documentados.
2. Evita monolitos Terraform: divide por dominios y ownership.
3. Usa naming conventions estrictas y etiquetas obligatorias.
4. Separa identidad/seguridad de workloads.
5. Automatiza revisiones de seguridad y coste antes del apply.
6. Minimiza privilegios de credenciales CI/CD (principio de menor privilegio).
7. Implementa estrategias de rollback y recovery de state.

30 Roadmap de estudio: de 0 a master

Fase 1 (Fundamentos, 1-2 semanas)

- HCL, variables, outputs, providers.
- `init/fmt/validate/plan/apply/destroy`.
- Proyecto simple: VPC + 1 instancia + bucket.

Fase 2 (Intermedio, 2-4 semanas)

- Módulos, `for_each`, data sources.
- Backend remoto y locking.
- Pipeline con `plan` en PR.

Fase 3 (Avanzado, 1-2 meses)

- Diseño multi-cuenta / multi-región.
- Seguridad IaC, políticas y compliance.
- Terratest, importaciones y migraciones complejas.

Fase 4 (Master, continuo)

- Arquitectura de plataforma interna reusable.
- Golden modules, guardrails y catálogo de patrones.
- Observabilidad de cambios y optimización de coste cloud.

31 Cheat Sheet final

```
# Inicialización y validación
terraform init
terraform fmt -recursive
terraform validate

# Plan y apply
terraform plan -out=tfplan
terraform apply tfplan

# Estado
terraform state list
terraform state show <address>

# Workspaces
terraform workspace list
terraform workspace select dev

# Limpieza
terraform destroy
```

Listing 28: Comandos que debes dominar

32 Glosario rápido (términos clave)

Término	Definición práctica
Provider	Plugin que conecta Terraform con APIs externas.
Resource	Objeto gestionado por Terraform (VM, red, bucket, namespace, etc.).
Data Source	Lectura de recursos existentes sin crearlos.
State	Base de verdad operativa de Terraform sobre recursos desplegados.

Plan	Vista previa exacta de cambios antes de aplicar.
Module	Componente reutilizable de infraestructura.
Workspace	Contexto de estado aislado dentro de una misma configuración.
Drift	Desviación entre lo declarado y lo que realmente existe.
GitOps	Operación declarativa continua desde Git para workloads en Kubernetes.

33 Conclusión

Terraform no es solo una herramienta, es una disciplina de ingeniería. El nivel *master* se alcanza cuando combinas:

- diseño modular sólido,
- operaciones seguras en equipo,
- automatización con CI/CD,
- y gobernanza de infraestructura a escala.

Si conviertes estos apuntes en práctica real con proyectos y revisiones técnicas, tendrás un dominio profesional de Terraform.